



Programmiersvorkurs

Einführung in C# - Tag 1

Ablauf



Montag - Donnerstag

Zeitraum	Was?
10 – 12 Uhr	Informationen, Übungsbesprechung, Vorlesung
12 – 13 Uhr	Mittagspause
ab 13 Uhr	Übungen, gemütliches Zusammensitzen

Inhaltsübersicht



Tag 1	Variablen, Datentypen, Arithmetik, Konvertierung, Zustände, Visual Studio Demo
Tag 2	Boolsche Ausdrücke, Kommentare, If-Abfragen, Switch-Case, Debugging
Tag 3	Arrays, (Do-)While-Schleife, For-Schleifen, Weiterführung Debugging
Tag 4	(statische) Methoden, Klassenvariablen, JavaDoc, Exceptions

Was ist Programmieren?

Das Schreiben von Anweisungen für einen Computer.

- Computer basieren auf einem binären System („0“ und „1“)
- Programmiersprachen sind für uns lesbar
- **Schnittstelle: Mensch und Computer**

Calculate
 $25 * 25$



??

Calculate
 $11001 * 11001$

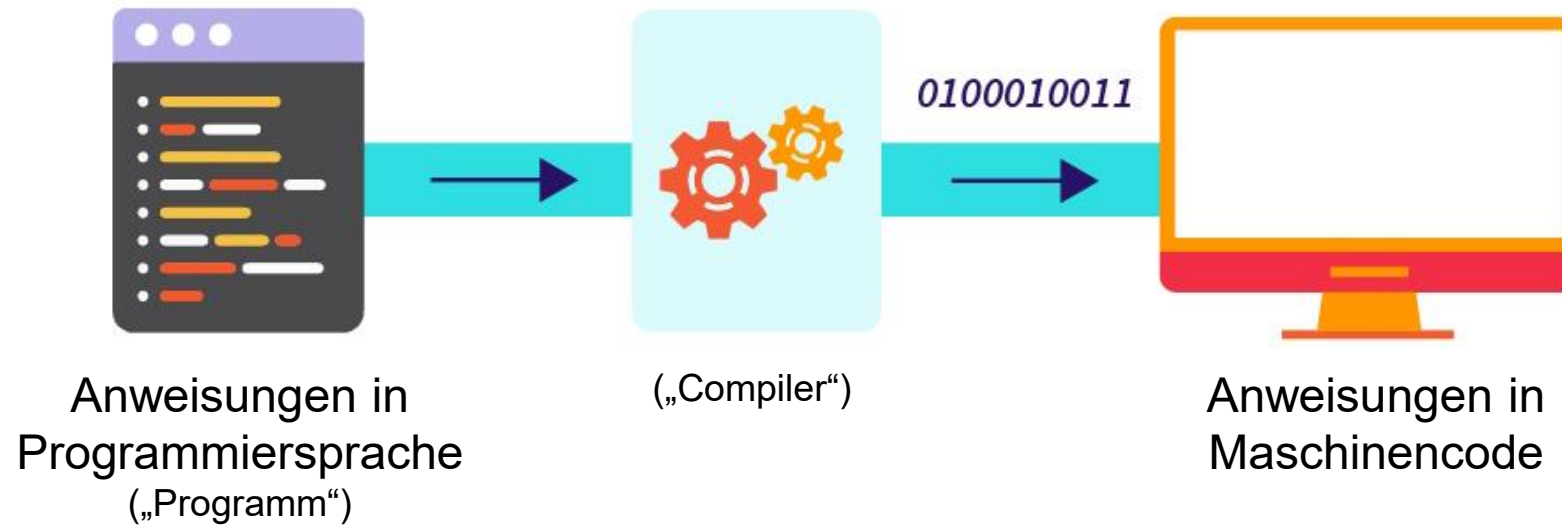


1001110001

Programmiersprache

Für uns lesbar:

Für Computer lesbar:



C#

WIKI

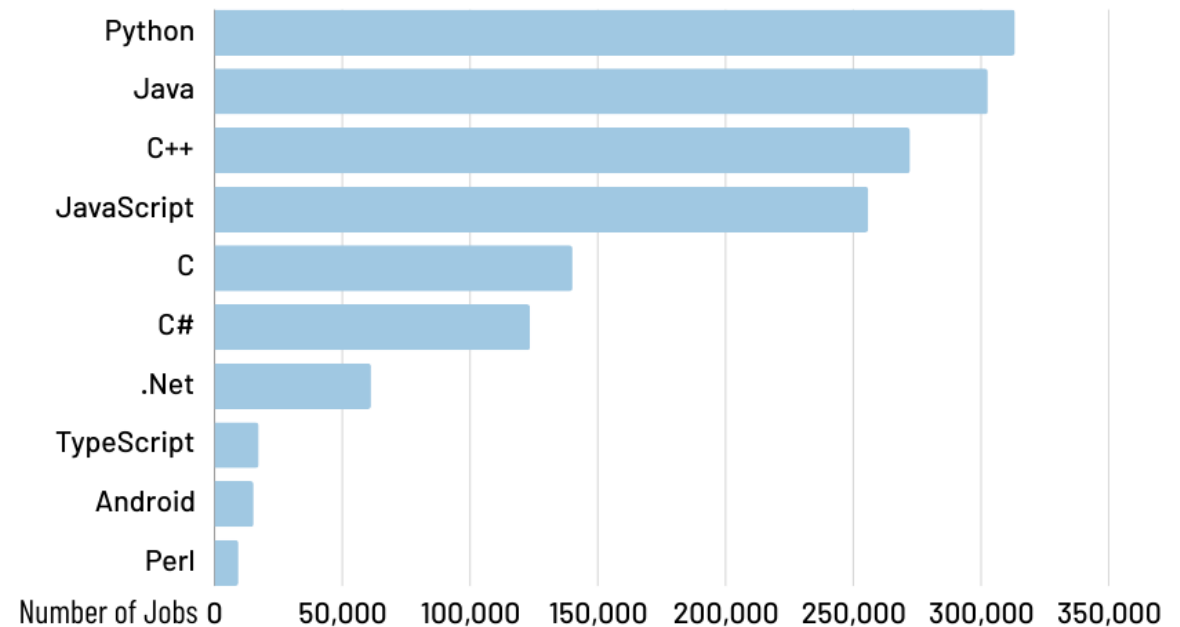


Motivation

- Eine der populärsten Programmiersprache weltweit!
- Einfach zu lernen/einfach zu benutzen

Most in-demand programming languages of 2024

Based on LinkedIn job postings in the US



By: CodingNomads

Anwendungsbereich



Haupt-Anwendungsbereiche:

- **Anwendungsentwicklung** (Windows)
- **Web-Anwendungen** und Web-Services
- **Games** (über z.B. Unity)



Hello World!



```
1 class Program
2 {
3     static void Main(string[] args)
4     {
5         Console.WriteLine("Hello World");
6     }
7 }
```

„Main-Methode“

Anweisung

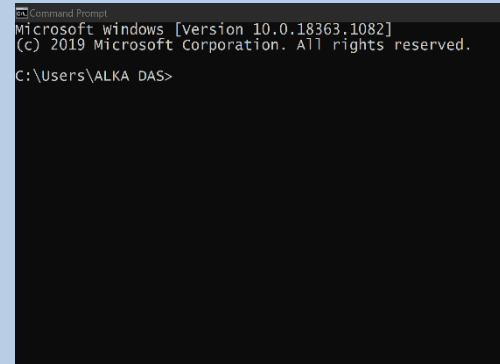
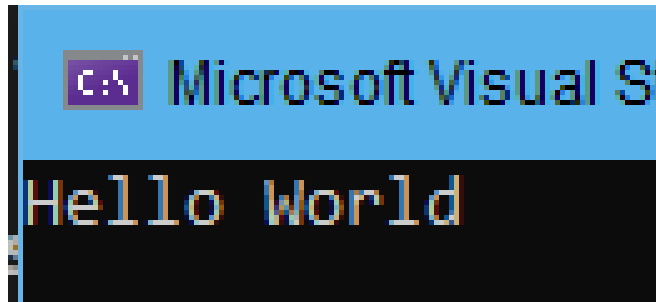
- **Startpunkt** für C# Programm: „static void **Main**“ (Main-Methode)
- Anweisungen (den Code) innerhalb der geschweiften Klammern der Main-Methode

Konsolen-Ausgabe

```
Console.WriteLine( "Hello World" );
```

- Schreibt **Text** in die Konsole
- Mit „“ markierter Text wird ausgegeben

Ergebnis:



Konsole = Text Ein-
/Ausgabe

Variablen

- Speicher für Werte, die sich ändern können
- Primitive Datentypen
 - Ganzzahlen
 - Fließkommazahlen
 - Wahrheitswerte
 - Zeichen
- Referenzdatentypen
 - Zeichenketten

Zahlen im Computer sind endlich!

- Ganzzahlen

- **byte** (0 to 255) (8 Bit / 1 Byte)
- **short** ($\pm 32,768$) (16 Bit / 2 Byte)
- **int** ($\pm 2,147,483,648$) (32 Bit / 4 Byte)
- **long** ($\pm 9,223,372,036,854,775,808$) (64 Bit / 8 Byte)

- Kommazahlen (Fließkommazahlen)

- **Float** (präzise auf ~6-9 Nachkommastellen) (32 Bit / 4 Byte)
- **Double** (präzise auf ~15-17 Nachkommastellen) (64 Bit / 8 Byte)
- **Decimal** (präzise auf ~28-29 Nachkommastellen) (128 Bit / 16 Byte)

- Wahrheitswerte
 - „**bool**“
 - **true** oder **false**
- Zeichen
 - „**char**“
 - Darstellung als 16-Bit-Unicode-Wert
- Zeichenketten (Text)
 - „**string**“
 - Referenzdatentyp

Variablen - Wertebereich

Type	Länge		Wertebereich
	Byte	Bit	
bool	–	–	true oder false
char	2	16	Unicode Zeichen
byte	1	8	–128 ...127
short	2	16	–32.768 ...32.767
int	4	32	–2.147.483.648 ...2.147.483.647
long	8	64	$-2^{63} \dots 2^{63} - 1$
float	4	32	$\pm 1,4^{E-45} \dots \pm 3,4^{E+38}$
double	8	64	$\pm 4,9^{E-324} \dots \pm 1,7^{E+308}$

Variablen

```
1 class Program
2 {
3     static void Main(string[] args)
4     {
5         string text = "Hello World!";
6
7         Console.WriteLine(text);
8     }
9 }
```

Variable: **text**

- string -> Variablen-Typ
- text -> Variablen-Name
- „=„ -> Zuweisung
- „Hello World“ -> Variablen-Wert
- ; -> Ende der Zeile

Variablen

```
1 class Program
2 {
3     static void Main(string[] args)
4     {
5         int number = 2;
6         int secondNumber = 4;
7
8         Console.WriteLine(number + secondNumber);
9     }
10 }
```

Typ Name = Wert;

- Danach ist der Name wie der Wert benutzbar

Variablennamen (Bezeichner)



- Vorgaben
 - Nur Buchstaben, Zahlen und _
 - Erstes Zeichen darf keine Zahl sein
 - Name muss eindeutig sein
 - Gesperrte Namen: z.B. „new“, „int“, „string“, ...

Variablennamen

- Konventionen

- So **SOLLTEN** Namen sein (aufgrund von Lesbarkeit)
- Sinnvoll, aussagekräftig
- Abkürzungen vermeiden
- Substantive

```
1 double payMoney;    // nicht aussagekräftig, kein Substantiv
2 double payTicket;   // kein Substantiv
3 double ticketPrice; // besser
```

Variablennamen

- Konventionen

- Nur lateinische Zeichen (normales Alphabet), Zahlen und _
 - Kein ä, ö, ü, ß, ...
- Englisch oder Deutsch (**nicht beides!**)
 - Üblicherweise Englisch
- lowerCamelCase-Schreibweise:

```
1 int animalCount;  
2 int bodyHeight;
```

Deklaration

- Allgemeine Deklaration einer Variable:

```
01  datentyp name;
```

- Beispiele:

```
1 int age;  
2 float weight;  
3 double height;  
4 bool isStudent;  
5 char gender;  
6 String name;
```

Zuweisung

- Allgemeine Zuweisung einer Variable:

```
01  datentyp name;  
02  name = Wert;
```

- Beispiele:

```
1  int age;  
2  age = 20;  
3  
4  float weight;  
5  weight = 72.3f;  
6  
7  double height;  
8  height = 178.3;  
9  //...
```

Deklaration + Zuweisung

- Allgemeine Deklaration einer Variable:

```
01  datentyp name;  
02  datentyp name = Wert;
```

- Beispiele:

```
1 int age = 20;  
2 float weight = 68.3f;  
3 double height = 178.62;  
4 bool isStudent = true;  
5 char gender = 'm';  
6 String name = „Max Mustermann“;
```

Typ Name = Wert;

Ausgabe

- Sonst wäre es ziemlich langweilig!

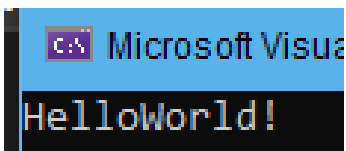
```
1 class Program
2 {
3     static void Main(string[] args)
4     {
5         string text = "Hello World!";
6
7         Console.WriteLine(text);
8     }
9 }
```

Verknüpfung

- Text lässt sich mit + Verknüpfen (Plus-Operator)

```
1 class Program
2 {
3     static void Main(string[] args)
4     {
5         string text = "Hello";
6         string textPart2 = "World!";
7
8         Console.WriteLine(text + textPart2);
9     }
10 }
```

- Ausgabe:

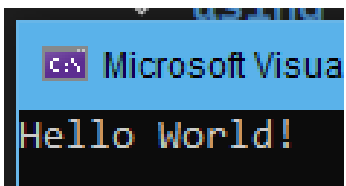


Verknüpfung

- Text lässt sich mit + Verknüpfen (Plus-Operator)

```
1 class Program
2 {
3     static void Main(string[] args)
4     {
5         string text = "Hello";
6         string textPart2 = "World!";
7
8         Console.WriteLine(text + " " + textPart2);
9     }
10 }
```

- Ausgabe:



Bezeichnung	Operator	Anwendung
Addition	+	$a + b$
Subtraktion	-	$a - b$
Multiplikation	*	$a * b$
Division	/	a / b
Inkrement	++	$a++$
Dekrement	--	$a--$
Modulo	%	$a \% b$

Das Ergebnis kann einer Variablen zugewiesen werden:

```
1 int result = 5 + 3;  
2 double division = 10.0 / (result - 1);
```

In- bzw. Dekrement

- Erhöht bzw. verringert den Wert einer Variable um 1

```
1 int scorePlayerOne = 42;  
2 int scorePlayerTwo = 38;  
3 scorePlayerOne++;  
4 scorePlayerTwo--;
```

scorePlayerOne ist jetzt 43
scorePlayerTwo ist jetzt 37

Bezeichnung	Operator	Anwendung
Addition	+	$a + b$
Subtraktion	-	$a - b$
Multiplikation	*	$a * b$
Division	/	a / b
Inkrement	++	$a++$
Dekrement	--	$a--$
Modulo	%	$a \% b$

Modulo (Restwertbestimmung)



- Das Ergebnis des Modulo ist der Rest der Division:

$$26/5 = 5 \text{ Rest } 1 \Rightarrow 26 \% 5 = 1$$

$$30/2 = 15 \text{ Rest } 0 \Rightarrow 30 \% 2 = 0$$

$$65/10 = 6 \text{ Rest } 5 \Rightarrow 65 \% 10 = 5$$

$$29/30 = 0 \text{ Rest } 29 \Rightarrow 29 \% 30 = 29$$

Bezeichnung	Operator	Anwendung
Addition	+	$a + b$
Subtraktion	-	$a - b$
Multiplikation	*	$a * b$
Division	/	a / b
Inkrement	++	$a++$
Dekrement	--	$a--$
Modulo	%	$a \% b$

Präinkrement

- Präinkrement (Prä → Davor)

```
1 int score = 23;  
2 Console.WriteLine(++score);
```

Wert wird *vor* der Ausgabe noch erhöht.

Bezeichnung	Operator	Anwendung
Addition	+	$a + b$
Subtraktion	-	$a - b$
Multiplikation	*	$a * b$
Division	/	a / b
(Prä-) Inkrement	++	$++a$
(Prä-) Dekrement	--	$--a$
Modulo	%	$a \% b$

Arithmetik Beispiele

Beispiel 1:

```
1 int itemCount = 5;  
2 itemCount = itemCount + 1;
```

Ist äquivalent zu:

```
1 int itemCount = 5;  
2 itemCount++;
```

Beispiel 2:

```
1 int totalPagesRead = 5;  
2 totalPagesRead = totalPagesRead + 5;
```

Ist äquivalent zu:

```
1 int totalPagesRead = 5;  
2 totalPagesRead += 5;
```

Integer-Division

- In C#:

$$5 / 2 = 2$$

Integer / Zahl = Integer

C# rundet Ergebnis ab!

Integer-Division

- Integer ist der Datentyp für Ganzzahlen
- Da `int` keine Kommazahlen darstellen kann, wird in C# bei einer Integer-Division **auf den nächstkleineren ganzen Wert abgerundet**

Beispiel:

```
1 int cakeSlices = 7;  
2 int guests = 2;  
3 double cakeSlicesPerGuest = cakeSlices / guests;
```

Ergebnis: cakeSlicesPerGuest ist 3.0
(nicht 3.5)

In diesem Kontext hätten wir das Ergebnis 3.5 erwartet, aber durch die Integer Division erhalten wir 3.0 und erzeugen damit ungenaue Informationen!

*Um das zu vermeiden, müssen wir korrekte **Typ-Konvertierungen** anwenden.*

Integer-Division

- „cakeSlices“ darf vom Typ keine Ganzzahl sein

Mögliche Lösung:

```
1 double cakeSlices = 7.0;  
2 int guests = 2;  
3 double cakeSlicesPerGuest = cakeSlices / guests;
```

Ergebnis: cakeSlicesPerGuest ist 3.5!

Implizite Konvertierung

- Einige Typen können ohne Probleme in andere umgewandelt werden

```
byte → short → int → long → double  
byte → short → int → float → double
```

```
1 int x = 42;  
2 float y = x;
```

```
1 byte a = 123;  
2 short b = a;  
3 long c = b;  
4 double d = c; //d = 123.0
```

- „Kleinere“ Datentypen können ohne Probleme in „größere“ Datentypen konvertiert werden
- C# konvertiert hier die Datentypen automatisch (**implizit**)

Explizite Konvertierung

- Variablenwerte können umgewandelt werden
→ „casten“

Beispiel 1:

```
1 int price = 42;  
2 double priceExact = (double)price;
```

Wir können C# hier **explizit** anweisen, die Variable `price` in den Datentyp `double` zu konvertieren

Welchen Wert beinhaltet priceExact?



priceExact: 42.0

Explizite Konvertierung

- Noch ein Beispiel:

```
1 double averageScore = 512.6;  
2 int roundedScore = (int)averageScore;
```

Welchen Wert beinhaltet roundedScore?



roundedScore: 512

Zurück zum Divisionsproblem

```
1 int cakeSlices = 7;  
2 int guests = 2;  
3 double cakeSlicesPerGuest = cakeSlices / guests;
```

- Mit casten:

```
1 int cakeSlices = 7;  
2 int guests = 2;  
3 double cakeSlicesPerGuest = (double)cakeSlices / guests;
```



cakeSlicesPerGuest: 3.5

- Die Variable `cakeSlices` wird hier explizit von `int` zu `double` konvertiert
- Die Division wird als Gleitkommaoperation durchgeführt, da einer der Operanden ein `double` ist

Zusammenfassung

- Allgemein gilt für die Division in C#:

Für Division mit Nachkommastellen muss der erste Operand eine Gleitkommazahl sein!

- z.B. durch casten:

```
1 double cakeSlicesPerGuest = (double)cakeSlices / guests;
```

Kommentare

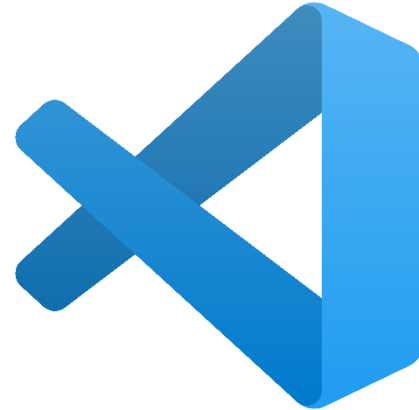
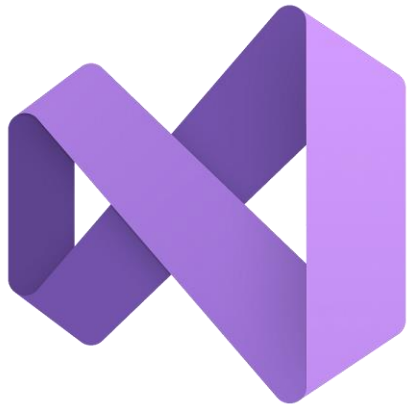
- Text im Code, der vom Programm ignoriert wird
- Syntax: `//beliebiger Text`
- Einzeilige Kommentare:

```
1 int number; //Kommentar
2
3 //number = 3;
4
5 number = 12;
```

Mehr zu Kommentaren an Tag 2!

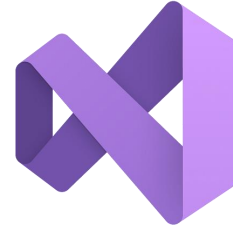
Entwicklungsumgebungen

- Wir Programmieren in einer IDE (Entwicklungsumgebung)



- Wir empfehlen in diesem Kurs die Nutzung von **Visual Studio**

IntelliJ Einführung



➤ Live Demo

Quellen und Mitwirkende



Original von Tim Roes
Überarbeitet 2012 von Christian Zimmer
Überarbeitet 2013 von Benedikt Haug
Überarbeitet 2016 von Jakob Ernst
Überarbeitet 2018 von Sebastian Glutsch
Überarbeitet 2019 von Marcel Rettig
Überarbeitet 2020 von Anja Buchmaier
Überarbeitet 2020 von Marcel Rettig
Überarbeitet 2021 von Tobias Hnyk
Überarbeitet 2025 von Simon Weinhardt